

HIVE tables:

```
Logging initialized using configuration in jar:file:/usr/hdp-2.6.0.0/hadoop-hdfs-client/lib/hive-log4j-core-1.2.17.jar!/log4j.properties
Hive Session ID = 8e58d7ca-3556-4e2a-972c-16f8e0e8
hive> show tables;
OK
content_metadata
dim_content
fact_user_actions
user_activity_logs
Time taken: 1.122 seconds, Fetched: 4 row(s)
hive> 
```

Table Descriptions

1. content_metada:

```
hive> describe content_metadata;
OK
content_id      string
title           string
category        string
Time taken: 1.404 seconds, Fetched: 3 row(s)
```

2. user_activity_logs:

```
hive> describe user_activity_logs;
OK
user_id          string
activity         string
event_timestamp  string
year             string
month            string
day              string

# Partition Information
# col_name      data_type      comp
year            string
month           string
day             string
Time taken: 0.34 seconds, Fetched: 12 row(s)
```

3. fact_user_actions:

```
CREATE TABLE IF NOT EXISTS fact_user_actions ( user_id STRING, action_type
STRING, timestamp TIMESTAMP, content_id STRING, action_details STRING )
PARTITIONED BY (year INT, month INT, day INT) STORED AS PARQUET;
```

```
hive> describe fact_user_actions;
OK
user_id          string
action           string
content_id       string
event_timestamp  timestamp
year             string
month            string
day              string

# Partition Information
# col_name      data_type      con
year            string
month           string
day             string
Time taken: 0.688 seconds, Fetched: 13 row(s)
```

4. dim_content:

```
CREATE TABLE IF NOT EXISTS dim_content ( content_id STRING, content_name  
STRING, content_type STRING, content_description STRING ) STORED AS  
PARQUET;
```

```
hive> describe dim_content;  
OK  
content_id          string  
title               string  
category            string  
published_date      timestamp  
Time taken: 0.093 seconds, Fetched: 4 row(s)
```

Queries for these tables:

```
WITH session_data AS (  
    SELECT  
        f.user_id,  
        f.year,  
        f.month,  
        f.day,  
        DATEDIFF(f.event_timestamp, LAG(f.event_timestamp) OVER (PARTITION  
BY f.user_id ORDER BY f.event_timestamp)) AS session_length_days  
    FROM  
        fact_user_actions f  
    WHERE  
        f.year = '2023' -- Modify year as needed  
        AND f.month BETWEEN '01' AND '12' -- Modify for specific months or  
a date range
```

```
SELECT  
    f.year,  
    f.month,  
    d.region,  
    COUNT(DISTINCT f.user_id) AS active_users  
FROM  
    fact_user_actions f  
JOIN  
    dim_content d  
ON
```

```

    f.content_id = d.content_id
WHERE
    f.year = '2023' -- Optional filter on year, modify as needed
    AND f.month BETWEEN '01' AND '12' -- Modify for specific months or a
    date range
GROUP BY
    f.year, f.month, d.region
ORDER BY
    f.year, f.month, active_users DESC;

```

Design choice and time complexity:

With these queries, I focused on aggregating user_actions on content, providing insights for active_user per region and play count per category.

Complexity-wise using "COUNT" can be computationally expensive and a little time consuming.

Design choice: (Major ones)

- Query-1:
 - LAG() window function, avoiding self-joins, which can be costly in large datasets.
 - Filtration using WHERE clause
 - Partitioning option like user_id
- Query-2:
 - Joins on content_id
 - Distinct COUNT
 - Indexing

Performance-wise:

- For both queries indexing made the computation a lot faster, and using "distinct count" we avoided counting duplicates, which comparatively is slow for large dataset. Still, not our case as our dataset was small.

REPO LINK: https://github.com/saadmuzammil098/HDFS_HIVE.git